

Linea Yield Manager Audit

OpenZeppelin Security - February 18, 2026

Summary

Summary

- **Type:** DeFi
- **Timeline:** From 2025-11-03 → To 2025-12-05
- **Languages:** Solidity

Findings

- Total issues: 44 (33 resolved, 11 unresolved)
- Critical: 0 (0 resolved)
- High: 1 (1 resolved)
- Medium: 7 (7 resolved)
- Low: 22 (14 resolved)

This site uses cookies to offer you a better user experience. Information about the cookies we use can be found in our Privacy Policy under "Cookie Policy". You can reject all non-essential cookies by choosing "Decline" or set your own preference under "Cookies settings".

[Cookies settings](#)

Accept

Decline

Scope

OpenZeppelin audited the [Consensys/linea-monorepo](#) repository at commit [4360e31f](#).

The audit included a [diff review](#) of the rollup contract upgrades from the [contract-audit-2025-01-06](#) branch and a full audit of the new yield management system.

In scope were the following files:

```
contracts/contracts
├─ LineaRollup.sol (diff audit)
├─ LineaRollupBase.sol (diff audit)
├─ LineaRollupYieldExtension.sol
├─ lib
│   └─ ErrorUtils.sol
│   └─ Math256.sol
│   └─ YieldManagerPauseManager.sol
└─ yield
    └─ LidoStVaultYieldProvider.sol
    └─ LidoStVaultYieldProviderFactory.sol
    └─ YieldManager.sol
    └─ YieldManagerStorageLayout.sol
    └─ YieldProviderBase.sol
    └─ libs
        └─ ValidatorContainerProofVerifier.sol
```

Update: We have reviewed all the submissions and have finalized the audit report, with commit [9c16ce3](#) being the final commit reviewed. Additionally, we have validated that the below Ethereum mainnet deployments of the contracts fully match the contracts in the last reviewed commit.

- YieldManager implementation: [0x751236A1aFC11B7F1A7630fe87b0Bd96AC5203C4](#)
- ValidatorContainerProofVerifier: [0x2309C45d44105928b483f608dD6140Fb65f3EBde](#)
- LidoStVaultYieldProviderFactory: [0xE4FC9F1A8cB97fEAd3C2b37c11AD5B1C2eF73959](#)
- LineaRollup implementation: [0x04728BF704a716C26F9EF4085013b760AC88563](#)

System Overview

The Linea Yield Manager enables the Linea bridge to earn yield on deposited ETH by staking it through external yield providers while maintaining sufficient reserves for user withdrawals. The system integrates with Lido's StakingVault infrastructure to stake ETH and mint liquid staking tokens (LST),

distributing generated yield back to L2 users.

Withdrawal Liveness Guarantee

A core invariant is that user withdrawals must never be permanently blocked. To ensure liveness, the system provides multiple withdrawal pathways:

- **Normal ETH withdrawal:** When sufficient ETH is available in the `L1MessageService` reserve, it can be withdrawn normally.
- **LST withdrawal:** When the `L1MessageService` reserve is depleted, users can withdraw Lido LST instead of ETH through `withdrawLST`.
- **Permissionless unstaking:** When the `L1MessageService` reserve falls below the minimum threshold, users can trigger beacon chain withdrawals.

This ensures that at least one viable withdrawal mechanism remains available under all conditions.

Architecture

The `YieldManager` contract uses a delegatecall-based adaptor pattern where yield provider implementations are called via delegatecall from the `YieldManager` context. This allows provider-specific logic to access and modify `YieldManager` storage directly. The current implementation includes a `LidoStVaultYieldProvider` adaptor for integration with Lido's vault system.

Key Components

- `YieldManager`: The core contract managing yield operations, reserve thresholds, and provider coordination
- **Provider Adaptors:** Implementation contracts called via delegatecall to handle provider-specific logic
- `LineaRollupYieldExtension`: An extension of the rollup contract that enables `withdrawLST` to mint LST when insufficient ETH is available and facilitates yield distribution to L2
- `ValidatorContainerProofVerifier`: Verifies beacon chain validator state using EIP-4788 for permissionless unstaking

Reserve Management

The system maintains two reserve thresholds in the `L1MessageService` (rollup) contract:

- **Minimum Reserve:** Critical threshold below which permissionless unstaking is enabled
- **Target Reserve:** Preferred level that the system attempts to maintain

When reserves fall below the minimum threshold, users can permissionlessly initiate beacon chain withdrawals after providing proofs of validator state. Privileged operators can also unstake and manage

reserves through role-based access controls.

LST Liability Tracking

When users withdraw during reserve deficits, they can receive Lido LST instead of ETH. The system tracks LST liabilities per provider, which are overcollateralized and lock funds in the Lido vault. LST liabilities are repaid when funds are available, with the locked amount preventing those funds from being withdrawn until repayment occurs.

Security Model and Trust Assumptions

This section outlines the trust assumptions identified during the audit.

Delegatecall Trust Model

The delegatecall-based adaptor architecture means that provider implementations execute with full access to `YieldManager` storage. As such, malicious or buggy adaptors can corrupt accounting and bypass access controls.

Provider adaptors are trusted system components that must be carefully audited before deployment. The `SET_YIELD_PROVIDER_ROLE` holder controls which adaptors are registered.

External Protocol Dependencies

The system relies on several external protocols operating correctly:

- **Lido StakingVault:** Handles ETH deposits, validator operations, and LST minting
- **Lido VaultHub:** Manages vault connections, liability tracking, and withdrawable amounts
- **EIP-4788 Beacon Roots:** Provides beacon chain state for permissionless unstaking proofs
- **Beacon Chain:** Validator withdrawals must eventually settle for pending amounts to clear

Other Design Features

Permissioned Unstaking Gas Optimization: The `unstake` function does not validate withdrawal requests or track pending amounts to save gas. Invalid requests only impact yield efficiency, not fund safety.

Pausable Permissionless Actions: Permissionless unstaking and reserve replenishment can be paused by the `PAUSE_NATIVE_YIELD_PERMISSIONLESS_ACTIONS_ROLE`. This allows privileged actors to halt permissionless operations during emergencies or system maintenance.

L2 Yield Distribution: Yield distribution to L2 users is managed by a separate contract on L2. The implementation details and distribution criteria are not part of the audit scope.

Privileged Roles

The `YieldManager` contract employs the following privileged roles:

- `DEFAULT_ADMIN_ROLE` : Manages all role assignments and controls system-wide access
- `YIELD_PROVIDER_STAKING_ROLE` : Transfers ETH to yield providers when withdrawal reserve is above minimum threshold
- `YIELD_PROVIDER_UNSTAKER_ROLE` : Initiates beacon chain withdrawals, withdraws ETH from providers, and manages reserve replenishment
- `YIELD_REPORTER_ROLE` : Reports earned yield and distributes it to L2
- `STAKING_PAUSE_CONTROLLER_ROLE` : Pauses and unpauses beacon chain staking per provider
- `OSSIFICATION_INITIATOR_ROLE` : Initiates the irreversible ossification process to shut down a provider
- `OSSIFICATION_PROCESSOR_ROLE` : Completes ossification and finalizes provider shutdown
- `WITHDRAWAL_RESERVE_SETTER_ROLE` : Updates minimum and target reserve thresholds
- `SET_YIELD_PROVIDER_ROLE` : Registers and removes yield provider adaptors` (critical trust boundary)
- `SET_L2_YIELD_RECIPIENT_ROLE` : Manages an allowlist of L2 addresses that can receive yield
- The system implements granular pause management through separate role pairs for pausing and unpausing different operations:
 - staking (`PAUSE/UNPAUSE_NATIVE_YIELD_STAKING_ROLE`)
 - unstaking (`PAUSE/UNPAUSE_NATIVE_YIELD_UNSTAKING_ROLE`)
 - permissionless actions (`PAUSE/UNPAUSE_NATIVE_YIELD_PERMISSIONLESS_ACTIONS_ROLE`)
 - donations (`PAUSE/UNPAUSE_NATIVE_YIELD_DONATION_ROLE`)
 - yield reporting (`PAUSE/UNPAUSE_NATIVE_YIELD_REPORTING_ROLE`)

High Severity

Duplicate Validator Requests DoS Permissionless Unstaking

The `unstakePermissionless` function does not track which validators have been used for unstaking requests. Thus, an attacker can submit multiple requests for the same validator using proofs from different slots or timestamps, each passing the validation. The global `pendingPermissionlessUnstake` counter is incremented for each request, but when the actual withdrawal arrives, only the real amount is decremented. The remaining inflated amount permanently blocks future permissionless unstaking across all providers due to the [global target deficit check](#).

For example, with a 200 ETH target deficit and 100 ETH withdrawable from a validator, an attacker submits two permissionless unstake requests for the same validator using proofs from different slots. Both proofs get verified successfully as the validator was active at both slots. The pending counter increases by 200 ETH, filling the deficit cap. When the actual withdrawal arrives, only 100 ETH is withdrawn and decremented, leaving 100 ETH permanently inflated in the pending counter. This blocks all future permissionless unstaking globally, requiring privileged intervention to fix. The attack is trivial to execute by back-running or copying legitimate permissionless unstake transactions without needing to generate proofs.

Consider updating the validator verifier contract to also prove the total pending withdrawals for a validator to enforce correct limits. Alternatively, per-provider pending caps based on user funds could be explored.

Update: Resolved in [pull request 1879](#). The review of the fix required an extended review due to comprehensive changes to the permissionless unstaking flow. The details of the extension can be found in the appendix.

Medium Severity

Zero-Amount Provider Withdrawal Blocks Partial Reserve Replenishment

The permissionless `replenishWithdrawalReserve` function allows anyone to top up the L1 withdrawal reserve when it falls below the minimum threshold. When the YieldManager's own balance is insufficient to cover the full deficit, the function computes a withdrawal amount as the minimum of the provider's withdrawable balance and the remaining deficit. However, when the provider has zero withdrawable value but the YieldManager holds some ETH, the computed `withdrawAmount` is zero and the function still attempts to withdraw from the provider.

The withdrawal call eventually reaches Lido's `StakingVault.withdraw`, which reverts when the amount is zero. This prevents partial replenishment during low-liquidity states when the provider has exhausted its immediately withdrawable funds but the YieldManager still holds ETH that could be used.

Consider skipping the provider withdrawal when `withdrawAmount` is zero and directly funding the

reserve with the available YieldManager balance.

Update: Resolved in commit [3211036](#) in [PR1853](#).

Incorrect Target Deficit Causes Unnecessary Staking Pause

The `_addToWithdrawalReserve` function augments the withdrawal reserve by combining the YieldManager's balance with the funds withdrawn from a yield provider. The function [passes the full reserve deficit](#) to the internal withdrawal logic without accounting for the YieldManager's balance that will also contribute to covering the deficit.

The `_withdrawWithTargetDeficitPriorityAndLSTLiabilityPrincipalReduction` function compares the withdrawal amount against the target deficit and pauses staking when the withdrawal amount is insufficient to cover it. Since it receives an overstated deficit, it incorrectly concludes that the withdrawal is insufficient, skips the LST payment, and pauses staking even when the combined funds fully cover the actual deficit.

For example, suppose the reserve deficit is 100 ETH, the YieldManager holds 60 ETH, and the function is called to add 100 ETH to the reserve. The contract needs to withdraw 40 ETH from the provider, which when combined with the YieldManager's 60 ETH exactly covers the 100 ETH deficit. However, the internal function compares the 40 ETH withdrawal against the full 100 ETH deficit, concludes that it is insufficient, and pauses staking unnecessarily.

Consider subtracting the YieldManager balance from the target deficit before passing it to the internal function.

Update: Resolved in commit [7d3839c](#) in pull request [1717](#) by moving the staking pause logic at the beginning of the function.

Forwarded ETH Inflates System Balance and Bypasses Reserve Deficit Gate

The `unstakePermissionless` function allows anyone to trigger validator withdrawals when the reserve is below the minimum threshold. While the function has been made `payable` to cover Lido's trigger fees, the forwarded ETH is credited to the YieldManager before the [deficit gate check](#).

The gate check uses `_getTotalSystemBalance` which includes the YieldManager's balance, now inflated by the forwarded value. Since the minimum reserve threshold is calculated as a percentage of total system balance, the forwarded value increases both the total and the threshold proportionally. This can make a reserve that was above the minimum appear to be below it, bypassing the gate. After passing the gate, the forwarded value is sent to Lido and any excess is refunded, making the attack economically viable.

Consider excluding the forwarded value from the system balance calculation when evaluating the minimum reserve for permissionless unstaking.

Update: Resolved in commits [3d27ce6](#), [2aaab97](#), [8a89343](#), and [674e4e1](#) in pull request [1873](#).

Staking Paused When Reserve Target Is Exactly Met

When [withdrawing from a yield provider to restore the reserve target](#), staking for that provider is paused if the withdrawn amount fails to meet the target. However, staking is also paused when the withdrawn

amount exactly equals the target deficit, even though the reserve has been successfully restored to its target level.

Consider only pausing staking when the withdrawn amount is strictly less than the target deficit, allowing normal operation to continue when the target is exactly met.

Update: Resolved in [PR 1717](#) at [commit 7d3839c](#). The team stated:

`_withdrawWithTargetDeficitPriorityAndLSTLiabilityPrincipalReduction` function is removed as of [5cbe6b5e](#)

As of [7d3839c](#), the permissioned withdrawal functions comply with the recommendation to “only pause staking when the withdrawn amount is strictly less than the target deficit”

LST Principal Repayment During Withdrawals Does Not Decrement User Funds

When withdrawing from a yield provider, the

`_withdrawWithTargetDeficitPriorityAndLSTLiabilityPrincipalReduction` function pays any outstanding LST principal liability and then [reduces the withdrawal amount](#) by the LST principal paid. This reduced withdrawal amount is passed to `_delegatecallWithdrawFromYieldProvider`, which [decrements the user funds by that amount](#). However, the withdrawal amount no longer includes the LST principal that was already paid and deducted from it. Since the internal LST repayment functions do not adjust user funds, the user funds are decremented insufficiently, causing them to be overstated by the LST principal amount paid.

This contrasts with the [funding flow](#), where the full amount is first sent to the provider, LST principal is then paid from the provider's balance, and the user funds are incremented only by the remaining amount after LST repayment.

Consider decrementing the user funds and system total by the LST principal amount paid when it occurs during withdrawals, matching the accounting logic used in the funding flow.

Update: Resolved at commit [5cbe6b5](#).

User Funds Not Decrementated When LST Is Withdrawn

When [withdrawing LST](#), the function checks that the total LST liability plus the new amount does not exceed the user funds on the provider, then calls the provider to mint the LST tokens. The [provider increments the LST liability principal](#) to track the debt, but the user funds are never decremented. Since LST withdrawals are an advance on user funds already on the provider (as noted in the code comment), the actual provider balance decreases by the amount withdrawn while the tracked user funds remain unchanged, causing them to be overstated by the LST amount withdrawn.

Consider decrementing the user funds and system total by the LST amount when withdrawing LST to accurately reflect that these funds have been used to mint LST tokens.

Update: Resolved in commit [5cbe6b5](#). The fix also affected yield reporting so an additional change in the yield calculation was made at commit [0e46ee6](#).

Reentrancy From Lido Vault Refund Address Callback

The Lido staking vault [calls the refund address](#) during withdrawals, giving execution control to an arbitrary external address. This occurs during the `unstakePermissionless` flow, creating a reentrancy vector. While no specific exploit vector was identified in the current implementation, similar risks may exist with other vendor contracts.

Consider adding reentrancy guards to external `YieldManager` functions, excluding the `receive` function avoid blocking withdrawals from vaults.

Update: Resolved in [PR 1889](#) at commit [ee9c63](#).

Low Severity

Pause Event Is Not Emitted Consistently

The `YieldProviderStakingPaused` event [is emitted](#) when the pause is directly triggered. However, it is not emitted when the pause is caused by the `_pauseStakingIfNotAlready` function, which is triggered when unstaking fails to eliminate the target deficit.

Consider emitting the `YieldProviderStakingPaused` event whenever staking is paused.

Update: Resolved in commit [b9871bb](#) in pull request [1750](#).

Provider Funding Event Is Emitted With Incorrect Arguments

The `YieldProviderFunded` event specifies the last two of its parameters as `userFundIncrement` and `1stPrincipalRepaid`. However, the event is emitted with its arguments passed in [the reverse order](#).

Consider swapping the order of the last two parameters either in the event declaration or emission.

Update: Resolved in [5cbe6b5](#).

Floating Pragma

Pragma directives should be fixed to clearly identify the Solidity version with which the contracts will be compiled.

Throughout the codebase, multiple instances of floating pragma directives were identified:

- `LidoStVaultYieldProvider.sol` has the `solidity ^0.8.30` floating pragma directive.
- `LidoStVaultYieldProviderFactory.sol` has the `solidity ^0.8.30` floating pragma directive.
- `YieldManager.sol` has the `solidity ^0.8.30` floating pragma directive.

Consider using fixed pragma directives.

Update: Resolved in commit [3fafb80](#) in pull request [1717](#).

Incomplete Docstrings

Throughout the codebase, multiple instances of incomplete docstrings were identified:

- In `LidoStVaultYieldProvider.sol`, the `exitVendorContracts` function, the `_yieldProvider` parameter is not documented.
- In `YieldManager.sol`, for the `minimumWithdrawalReservePercentageBps`, `minimumWithdrawalReserveAmount`, `targetWithdrawalReservePercentageBps`, and `targetWithdrawalReserveAmount` functions, not all return values are documented.

Consider thoroughly documenting all functions/events (and their parameters or return values) that are part of a contract's public API. When writing docstrings, consider following the [Ethereum Natural Specification Format](#) (NatSpec).

Update: Resolved in commit [83c88b1](#) and [0e1c121](#) in pull request [1717](#).

Missing Docstrings

Throughout the codebase, multiple instances of missing docstrings were identified:

- In `Math256.sol`, the `Math256` library
- In `YieldProviderBase.sol`, the `YIELD_MANAGER` state variable

Consider thoroughly documenting all functions (and their parameters) that are part of any contract's public API. Functions implementing sensitive functionality, even if not public, should be clearly documented as well. When writing docstrings, consider following the [Ethereum Natural Specification Format](#) (NatSpec).

Update: Resolved in commit [3415368](#) in pull request [1717](#).

LST Withdrawal Can Be Stuck When Single Provider Lacks Liquidity

When claiming a message and [withdrawing LST](#), the user must specify a single yield provider to mint the full withdrawal amount. The withdrawal [reverts if that provider's user funds are insufficient](#) to cover the amount plus the existing LST liability. For large withdrawals, a provider that had sufficient liquidity when the user submitted the withdrawal message may no longer have it when the message is claimed on-chain. Since the user cannot split the withdrawal amount across multiple providers after message submission, the withdrawal becomes stuck until that specific provider regains sufficient liquidity.

Consider allowing users to supply an array of yield providers with corresponding amounts that add up to the total withdrawal value. Doing so will allow withdrawals to be fulfilled across multiple providers even when no single provider has sufficient liquidity.

Update: Acknowledged, not resolved. Linea team stated:

Operational mitigations: - We only plan to use a single YieldProvider for the time being - We will run a backend automation service to regularly rebalance funds such that the `L1MessageService` doesn't fall into a sustained deficit

Concerns: - We'd prefer not to break the "single call, single YieldProvider" semantics. Bundling multiple YieldProvider (and hence different staking vendor contract systems) into a single call will increase the complexity significantly.

Reserve Funding Function Vulnerable to DoS if Called With Zero Value

The `_fundReserve` internal function calls the rollup contract's `fund()` function, which **reverts if the sent value is zero**. While current call sites handle this appropriately, in future code iterations, calling this function without checking for a zero value first could introduce a DoS state where operations fail unexpectedly.

Consider removing the zero-value check in the rollup contract's `fund()` function to allow zero-value calls to succeed harmlessly. Doing so will make the function safer to use without requiring all callers to perform explicit zero-value checks.

Update: Acknowledged, will resolve.

Permissionless Unstake Cap Ignores System-Wide Liquidity

The `unstakePermissionless` function is designed to cap permissionless unstake requests to the remaining target deficit that cannot be covered by other liquidity sources. However, when **calculating available funds**, the implementation only considers the withdrawable value from the specified yield provider, not from all registered providers. This allows a caller to target a provider with low or zero withdrawable value and pass the cap check even when other providers have sufficient immediate liquidity to cover the entire deficit. The result is unnecessary validator exits that reduce yield efficiency, as funds are being unstaked from the beacon chain when they could have been withdrawn from other providers immediately.

Consider whether the tradeoff between yield efficiency and implementation complexity is acceptable, or if system-wide liquidity should be accounted for through alternative mechanisms.

Update: Acknowledged, not resolved. The Linea team stated:

The worst outcome here is that excess `unstakePermissionless` is permitted. This is acceptable as it prioritizes our core goal - censorship-resistant user withdrawals - over yield efficiency.

Iterating across the `withdrawableValue()` call for each yieldProvider (an unbounded array) can be problematic and introduce a DOS vector.

Here we are given the choice between being more lenient than required for `unstakePermissionless`, and being stricter than required - we prefer to be more lenient

given it aligns with our core goal and enables simpler implementation.

Fallback Operator Cannot Renounce Any Role

The [interface documentation](#) states that the `OnlyNonFallbackOperator` error is thrown when the fallback operator tries to renounce their operator role. However, the `renounceRole` [implementation](#) reverts for any role when the caller is the fallback operator, not just the operator role. This prevents the fallback operator from voluntarily renouncing other roles they may hold such as verifier setter or admin roles, even though role admins can still revoke those roles.

Consider restricting the check to only prevent renouncing the operator role specifically, allowing the fallback operator to renounce other roles as intended by the interface documentation.

Update: *Acknowledged, will resolve. Linea team stated:*

Operationally we will only provide the single OPERATOR_ROLE role to the fallbackOperator address.

Incorrect Argument Order in Finalization State Error

The `FinalizationStateIncorrect` error is defined with the signature `error FinalizationStateIncorrect(bytes32 expected, bytes32 value)`, indicating that the expected value should come first. In `setFallbackOperator`, the error is called with arguments reversed, passing the actual value first and the expected value second.

Consider swapping the argument order to match the error signature and the correct usage in other locations.

Update: *Acknowledged, will resolve.*

Funding Allowed During Pending Ossification

During the ossification process, `withdrawLST` and `payLSTPrincipal` block operations when either ossification is initiated or completed. However, `fundYieldProvider` only checks if ossification is completed, not if it has been initiated. This allows privileged funding to continue during pending ossification, creating an inconsistency where LST operations are frozen but staking operations can still occur through the privileged role.

Consider adding an `isOssificationInitiated` check to `fundYieldProvider`.

Update: *Resolved in commit [5cbe6b5e](#) in pull request [1717](#).*

Node Operator Fees Not Disbursed When Balance Exactly Equals Fee

When paying node operator fees, the function [checks if the available vault balance is strictly greater than the accrued fees](#) before disbursing. This prevents payment when the available balance exactly

equals the fees, even though there are sufficient funds for a full payment. As a result, the fees remain unpaid and continue to accrue until the balance exceeds the fee amount.

Consider changing the condition to use a greater-than-or-equal-to comparison to allow payment when the balance exactly matches the accrued fees.

Update: Resolved in commit [0e46ee6](#) in pull request [1717](#) by unconditionally attempting node operator fee payment.

Provider Events Do Not Include Yield Provider Address

When [unstaking from yield providers](#), provider adapters emit events via delegatecall, logging them from the yield manager's address. However, these [provider-specific events](#) do not include the identifying yield provider address that was passed to the YieldManager function. When multiple yield providers are registered, this makes it ambiguous which provider emitted the event.

Consider emitting standardized events at the YieldManager level. These events should include the yield provider address, with an optional bytes parameter for provider-specific data to maintain flexibility while providing a consistent interface for event consumers.

Update: Resolved in commit [b8e142f](#) in pull request [1717](#).

Incorrect Check When Unpausing Staking

The [unpauseStaking](#) [documentation](#) states that the function will revert if ossification has been initiated or finalized. However, the [implementation only checks for pending ossification](#), allowing the call to proceed when ossification is finalized.

Consider correcting the ossification check to also revert when ossification is finalized.

Update: Resolved. Linea team stated:

This check is on the [LidoStVaultYieldProvider::unpauseStaking](#) function.

LST Withdrawal Allowed When Balance Exactly Matches Withdrawal Amount

The [LST withdrawal function](#) checks that a deficit exists before allowing LST withdrawal by reverting if the withdrawal value is less than the contract balance. However, when the balance exactly equals the withdrawal value, there is no deficit, but the check passes and LST withdrawal is allowed.

Consider using a less-than-or-equal-to comparison.

Update: Resolved in commit [9722a2a](#) in pull request [1717](#).

Reserve Funding Skips LST Liability Payment When Manager Has Sufficient Balance

When [adding to the withdrawal reserve](#), if the YieldManager balance is sufficient to cover the amount, the function directly funds the reserve without paying any LST liability. However, when the manager balance is insufficient and funds must be [withdrawn from the provider](#), LST liability is paid from the withdrawn amount. This creates inconsistent behavior where LST liabilities may remain unpaid when the manager has available funds.

Consider documenting this design choice in the code comments to clarify that LST liability payment is skipped for operational efficiency when using manager balance.

Update: Resolved in in commit [0d00ee4](#) in pull request [1717](#) by clarifying the rationale for the asymmetric behavior in the comments.

Permissionless Unstake Can Be Griefed With Small Donation

The `unstakePermissionless` function reverts if the available funds plus the unstake amount exceed the target deficit. A griefing attacker can cause a user's unstake transaction to revert by donating a small amount to the contract, forcing the user to recalculate and resubmit with a lower unstake amount.

Consider clamping the unstake amount to the remaining deficit instead of reverting.

Update: Acknowledged, not resolved. Linea team stated:

We acknowledge the identified griefing vector. The current architecture intentionally separates responsibilities between the `YieldManager` and individual `YieldProvider` implementations, and the recommendation would require breaking these design boundaries.

- The **YieldManager** does not parse or interpret the unstake amounts encoded within the provider-specific payload.
- The **YieldProvider** does not have insight into the withdrawal reserve state, which is solely managed by the YieldManager.

[In the described attack scenario], the affected user can call `replenishWithdrawalReserve`, which effectively transfers the attacker's donation to the `L1MessageService`. This mechanism immediately increases available withdrawal liquidity and mitigates the impact of the griefing attempt.

Permissionless Unstaking Operational Limitations

The `unstakePermissionless` function has two operational limitations. First, it is only callable when the reserve is below the minimum threshold. A user can choose to unstake an amount that brings the reserve just above the minimum but far below the target, blocking further permissionless unstaking and requiring privileged roles to bridge the gap to the target reserve level.

Second, users select which yield provider to unstake from. When multiple providers exist with different unstaking speeds, a user can initiate unstaking from a slower provider, blocking further permissionless unstaking from faster providers until the reserve reaches the minimum.

Consider allowing permissionless unstaking until the target reserve is reached and implementing admin-configurable priority ordering for provider selection. However, these improvements should be weighed against the increased code complexity.

Update: Acknowledged, not resolved. Linea team stated that the behavior is intentional for the first case, and the fix violates their current design principals for the second case.

Emergency Removal Does Not Decrement Global User Funds Total

When [removing a yield provider](#), the provider storage is deleted without first decrementing `userFundsInYieldProvidersTotal` by an amount equal to the provider's `userFunds`. This leaves the global total permanently overstated, inflating the system balance calculation used for reserve thresholds and gating decisions.

Consider decrementing `userFundsInYieldProvidersTotal` by the provider's `userFunds` before deleting the provider storage.

Update: Resolved in [PR 1876](#).

Validator Proof Verifier Parameters Are Immutable

The `ValidatorContainerProofVerifier` [uses immutable parameters](#) for fork-specific generalized indices and a [single pivot slot](#) to select between pre-fork and post-fork beacon state tree structures. While this design accommodates one fork transition (if a future fork is known at contract deployment), future consensus layer forks that restructure the beacon state would require deploying a new verifier with updated parameters. Since the [yield provider's verifier reference is also immutable](#), this could cause a DoS for permissionless unstaking until the provider is upgraded or a new provider is deployed.

Consider making the `ValidatorContainerProofVerifier` constructor parameters changeable through a governance-controlled function to accommodate future fork transitions seamlessly without requiring redeployment.

Update: Acknowledged, will resolve.

Zero Amount Withdrawal Allows Unauthorized Pausing

Calling `withdrawFromYieldProvider` [with zero amount](#) when the reserve is at or above the target causes [staking to be paused](#) because the available funds for LST liability payment are zero. This allows holders of `YIELD_PROVIDER_UNSTAKER_ROLE` to pause staking without holding the `STAKING_PAUSE_CONTROLLER_ROLE`. While the current Lido implementation reverts on zero withdrawals, other yield provider implementations may not, enabling this privilege escalation.

Consider adding an early revert when the withdrawal amount is zero to prevent this unintended pausing behavior.

Update: Resolved at commit [7d3839](#).

Misleading Names And Comments

There are multiple misleading names and comments within the contracts.

- The `ErrorUtils` [title comment](#) refers to the wrong library.
- The ["Slot 1" comment](#) appears to be referring to all subsequent variables, including variables in slot 2, 3, and 4.
- This [@param statement](#) should be a `@return` statement.
- The `unstake` [comment](#) claims the refund address can be zero, but the `StakingVault` [would revert](#).
- [These errors](#) state the target must be above the minimum, but it can equal the minimum. The names would be more accurate if they were `TargetReservePercentageCannotBeBelowMinimum` and `TargetReserveAmountCannotBeBelowMinimum`.
- The Natspec comments in the `LidoStVaultYieldProvider` and the `ICommonVaultOperations` state that the zero address is a sentinel value for the caller address. However, the actual behavior of the Staking Vault is to [revert](#) on zero address refund recipient.
- The Natspec comment for the amount parameter [states](#) that the amount is in `wei`, when in fact it is in `gwei`.
- The `safeAddToWithdrawalReserve` caps the rebalance amount to provider withdrawable plus YieldManager balance, but the comment states it only [caps to withdrawable value](#).
- [Typo in the comment](#): "validate" instead of "validate".

Consider updating the comments and names to be aligned with the code.

Update: Resolved in [PR 1890](#).

Notes & Additional Information

Constant Not Using `UPPER_CASE` Format

In `YieldManagerStorageLayout.sol`, the `YieldManagerStorageLocation` [constant](#) is not declared using `UPPER_CASE` format.

According to the [Solidity Style Guide](#), constants should be named with all capital letters with underscores separating words. For better readability, consider following this convention.

Update: Acknowledged, not resolved.

State Variables Are Initialized by a Function

Special care must be taken when initializing state variables through an immediate function call so as not to incorrectly assume that the state is initialized. The same function can set different values for state variables because it would check a state variable that is not yet initialized.

Within the `ValidatorContainerProofVerifier` contract, multiple instances of state variables being

initialized by a function were identified:

- The `GI_VALIDATOR_CONTAINER_ROOT` state variable
- The `GI_STATE_ROOT` state variable

To improve the overall clarity, intentionality, and readability of the codebase, consider avoiding initializing a state variable via a function. If variables must be set upon contract deployment, perform the initialization in the constructor instead.

Update: Acknowledged, not resolved. Linea team stated:

The state variable initialisation pattern originates from the original Lido `CLProofVerifier` implementation. We'd prefer to retain this pattern for consistency with the original audited codebase.

Unaddressed TODO Comments

During development, having well described TODO/Fixme comments will make the process of tracking and solving them easier. However, these comments might age and important information for the security of the system might be forgotten by the time it is released to production. These comments should be tracked in the project's issue backlog and resolved before the system is deployed.

Throughout the codebase, multiple instances of unaddressed TODO/ `@dev` comments were identified:

- The TODO comment in line 38 of `LineaRollup.sol`.
- The `@dev` comment in line 26 of `LineaRollupYieldExtension`

Consider removing all instances of TODO/Fixme comments and instead tracking them in the issues backlog. Alternatively, consider linking each inline TODO/Fixme to the corresponding issues backlog entry.

Update: Resolved in commit `d8f57d5`.

Avoidable Using Literals

Within the `ValidatorContainerProofVerifier` contract, a sentinel value for a block in the far future is assigned as `18446744073709551615`. This value is equal to the maximum value of `uint64` type.

To improve readability, consider updating the literal value with the built-in `type(uint64).max`.

Update: Resolved in commit `44c4d14`.

Lack of Indexed Event Parameters

Throughout the codebase, multiple instances of events not having any indexed parameters were identified:

- The `LidoStVaultYieldProviderDeployed` event of `LidoStVaultYieldProvider.sol`
- The `LidoStVaultYieldProviderFactoryDeployed` event of `LidoStVaultYieldProviderFactory.sol`

To improve the ability of off-chain services to search and filter for specific events, consider [indexing event parameters](#).

Update: Acknowledged, not resolved. Linea team stated:

[The two identified events] are not intended to be consumed or filtered by off-chain services in normal operation.

State Variable Visibility Not Explicitly Declared

Within `YieldManager.sol`, the `MAX_BPS` state variable lacks an explicitly declared visibility.

For improved code clarity, consider always explicitly declaring the visibility of state variables, even when the default visibility matches the intended visibility.

Update: Resolved in commit [86db244](#) in pull request [1717](#).

Missing Bounds Check in Yield Provider Index Query

The `yieldProviderByIndex` function directly accesses the array without bounds checking. When an out-of-range index is provided, this causes a generic panic instead of reverting with the custom `YieldProviderIndexOutOfBounds` error defined in the interface.

Consider adding explicit bounds checking to revert with the custom error when the index is out of range.

Update: Resolved in commit [8b289da](#) in pull request [1856](#).

Pending Permissionless Unstake Not Decrementated on Donations

The [documentation states](#) that `pendingPermissionlessUnstake` will be greedily reduced with donations or withdrawals from the yield provider. However, when [donations are received](#), the counter is not decremented.

Consider decrementing `pendingPermissionlessUnstake` in the `receive` function instead of in the `_delegatecallWithdrawFromYieldProvider` function. Alternatively, consider updating the documentation to reflect the current behavior.

Update: Resolved in commit [50af7a9](#) in pull request [1873](#).

Uninitialized Proxy Can Be Frontrun

The `initialize` function is external and can be called by anyone on an uninitialized proxy. The first caller to initialize an uninitialized proxy would gain control of the admin role and all privileged roles.

Consider ensuring that the proxy is atomically initialized during deployment and performing proper checks before using the contract in production.

Update: Resolved. The deployment will use the recommended practices.

Code Quality Improvements

Throughout the codebase, multiple opportunities for code quality improvement were identified:

1. The **zero address check** in `initialize` could use the **dedicated utility function** for improved consistency.
2. The **checks** in `onlyKnownYieldProvider`, `isYieldProviderKnown`, and `addYieldProvider` all verify the same condition. Consider extracting this logic into a shared internal function to reduce code duplication.
3. The **zero-value check** in `_decrementPendingPermissionlessUnstake` is unnecessary as the next line using `safeSub` will handle the zero-value case correctly.

Consider applying the above-listed improvements to enhance code maintainability and reduce duplication.

Update: Resolved in commit [b4b8ef5](#) and in commits [094644c](#) and [50a5da1](#) in pull request [1888](#).

Client Reported

LST Liabilities Lock Funds and Reduce Withdrawable Value

When LST is minted from a Lido StakingVault, the LST liability plus a reserve amount is **marked as locked** in `VaultHub`. The vault's **withdrawable value** is capped by the unlocked amount, making funds backing LST liabilities unavailable for withdrawal. This locked amount cannot be withdrawn until the LST liability is repaid, as attempting to withdraw locked funds will result in the vault returning less than the requested funds.

Consider ensuring that withdrawal calculations and reserve deficit computations account for the fact that LST liabilities effectively reduce the withdrawable balance of the provider.

Update: Resolved in [pull request 1849](#).

Cannot Complete Ossification When There Is No Staged ETH

The Lido StakingVault's `unstage` function reverts when called with an amount of zero. During the ossification process, the contract **calls** `unstage` to withdraw any staged deposits. If there are no staged deposits, this call would revert and block the completion of the ossification process.

Consider skipping the `unstage` call if the amount is zero.

Update: Resolved in [PR 1877](#) at [commit bc1523b](#).

Lido StakingVault Reverts on Zero Amount When Unstaging

During `unstage`, Lido StakingVault checks if the input amount is zero and [reverts if that is the case](#). Within Linea's `LidoStVaultYieldProvider`, the `unstage` function is [called during ossification](#). This behaviour can prevent the progression of the ossification if the StakingVault has nothing to unstage.

Consider skipping the `unstage` call if the StakingVault has no staged balance.

Update: Resolved in [pull request 1877](#).

`LidoStVaultYieldProvider` Reports Inflated Yield Amount

The Lido StakingVault requires a 1 ETH `CONNECT_DEPOSIT` upon creation. This value is [added](#) to the vault and included in the [vault's total value](#). In the context of Linea's `LidoStVaultYieldProvider`, this value is not part of user yield, but a safety deposit owed to the Linea admin. Therefore, inclusion of this value within the `reportYield` function incorrectly affects contract accounting.

Consider excluding the `CONNECT_DEPOSIT` amount from the reported yield amount.

Update: Resolved in [pull request 1961](#).

Conclusion

This audit examined the Linea Native Yield System and associated bridge contract modifications. The Linea Yield Manager enables the Linea bridge to earn yield on deposited ETH by staking it through external yield providers while supporting user withdrawals. The system integrates with Lido's `StakingVault` infrastructure to stake ETH and mint liquid staking tokens (LST), distributing generated yield back to L2 users. The Native Yield System introduces a yield management layer between Linea's L1 bridge and external yield-generating protocols.

The `YieldManager` contract serves as the central coordinator, maintaining withdrawal reserve thresholds, tracking user funds deployed across yield providers, and orchestrating yield reporting to L2. It interacts with yield provider adaptors via `delegatecall`, allowing vendor-specific logic to execute within `YieldManager`'s storage context while preserving a unified accounting model. The `LidoStVaultYieldProvider` adaptor allows `YieldManager` to interact with Lido's V3 staking infrastructure, specifically the `Dashboard` and `StakingVault` contracts.

The audit identified one high-severity issue related to duplicate validator requests that could DoS permissionless unstaking. Multiple medium-severity vulnerabilities were also reported, which included accounting bugs where LST withdrawals failed to decrement user funds and incorrect target deficit calculations, causing unnecessary staking pauses. Several low-severity findings addressed edge cases in reserve management and pause logic, while notes covered code-quality improvements. The audited scope changed significantly over the course of the engagement as the team iterated on the design.

The Linea team responded quickly and thoroughly to all the findings and kept submitting fixes throughout the engagement. They supplied the audit team with detailed documentation and proactively shared their own issues and concerns during the audit.

Appendix

Appendix - Fix Review Extension

The fix for H-01 involved considerable changes, which required additional review time. This review was performed from December 29, 2025, to January 5, 2026. The fix review was primarily focused on [pull request 1879](#), which includes the fixes for H-01 and L-20. In this timeframe, we also reviewed the fixes for CR-03 and CR-04. Finally, we compared all the reviews done so far to [pull request 1965](#), which included combined fixes from all the reviews. During the review extension, we did not find any major issues with the code; however, there were a few potential code improvements, which are listed below.

CLZ Opcode Can be Used

The `bitLength` function and `fls` function are two implementations of the same operation. Consider removing one of them for simplicity.

Alternatively, if the codebase can be updated to Solidity 0.8.31, consider using the new `clz` opcode instead.

Update: Resolved in commits [3e86dad](#) and [57732a9](#) in pull request [1965](#).

Misleading Documentation

We have identified the following instances of misleading documentation: - [The comment](#) claiming the `BeaconTypes` were copied from a particular Lido commit is no longer accurate. - The `_pubkeys` variable now corresponds to exactly one pubkey.

Consider updating these comments accordingly.

Update: Resolved in commit [43df9a0](#) in pull request [1965](#).

Unsafe Cast

The unstaked amount is [reduced to 64 bits](#) with an unsafe cast. Although this will not be a problem in practice, consider including an explicit bounds check for robustness.

Update: Acknowledged. The team stated:

`unstakeAmountGwei` has a maximum value of 2016000000000 (2016 ETH worth of gwei) [per consensus specification]. Hence an additional bounds check for uint64 conversion should be redundant.

Mark Assembly as Memory-Safe

[This assembly block](#) respects the memory model so it can be marked as memory-safe to facilitate compiler optimizations.

Update: Resolved in commit [2a5cd1c](#) in pull request [1965](#).

Use `toLittleEndian64` Where Possible

Since `count` is at most 28 bits, consider replacing the `toLittleEndian` invocation with the optimised `toLittleEndian64` function version.

Update: Resolved in commit [8b98bc0](#) in pull request [1965](#).

Redundant Path Concatenation

The generalized index of the `Validator` container is the [concatenation](#) of

- the path from the beacon block root to the state root
- the path from the state root to the indexed validator node root
- the path from the validator root to the relevant validator field

However, in the current version of the codebase, the target leaf [represents the entire Validator object](#), so the last step is redundant. Consider removing it.

Update: Resolved in commit [62818e8](#) in pull request [1965](#).

Ready to secure your code?

[Request an Audit→](#)

Smart Contracts

Contracts Libraries

Contracts Wizard ↗

Contracts MCP ↗

Upgrades Plugin ↗

Open Source Tools

Company

About Us

Why OpenZeppelin

News

Customer Stories

Careers

Brand Kit

[Relayer](#)

[Monitor](#)

[UI Builder ↗](#)

[Safe Utils ↗](#)

[Role Manager ↗](#)

[The Open Source Stack](#)

Blockchains

[Arbitrum](#)

[Base](#)

[Ethereum](#)

[Linea](#)

[Midnight](#)

[OP Mainnet](#)

[Polkadot](#)

[Starknet](#)

[Stellar](#)

[Zama](#)

[ZKsync](#)

[Trust ↗](#)

[Terms & Policies](#)

[Terms of Service](#)

[Privacy Policy](#)

[Trademark Guidelines](#)

[AGPL License](#)

Services

[Smart Contract Security Audit](#)

[Blockchain Infrastructure Audit](#)

[ZKP Audit](#)

[Security Services](#)

Solutions

[Ecosystem Stack](#)

[Financial Institutions Stack](#)

Resources

[Research](#)

[Documentation ↗](#)

[Stats](#)

[Forum ↗](#)

[Ethernaut CTF ↗](#)

